SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5116

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Assessment Tool Weightage**:** 10%

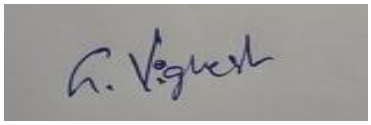
QUESTIONS: **Total Marks: 50**

Annexure A

Debugging & Optimization Task

Code of Conduct:

*I **Gullapelly Vignesh** (192373051) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*



Signature of the student:

Annexure A
Debugging & Optimisation Task

1. Debugging & Optimization of Symmetric Encryption (DES / 3DES Implementation)
A Python program implementing **DES and 3DES encryption** produces inconsistent ciphertext for identical inputs across multiple runs.

A. Identify and fix logical and implementation errors in key scheduling and permutation steps.
B. Optimize the code to reduce execution time for large input datasets (≥10 MB).
C. Justify whether replacing DES/3DES with a modern cipher improves both security and performance.

2. Performance Analysis of Block Cipher Modes of Operation A Python script implements **AES-like block cipher modes (ECB, CBC, CFB, OFB)** but suffers from high latency and memory inefficiency.

A. Debug incorrect padding and IV handling causing decryption failures.
B. Refactor the code to minimize redundant computations and memory usage.
C. Compare and optimize the implementation to determine the most efficient mode for real-time secure communication.

3. Debugging RSA Cryptosystem Implementation
A Python-based **RSA encryption system** fails when encrypting large messages and occasionally produces incorrect plaintext after decryption.

A. Identify issues in prime generation, key generation, and modular exponentiation.
B. Optimize the algorithm using efficient techniques (e.g., fast exponentiation, Chinese Remainder Theorem).
C. Evaluate the trade-offs between key size, security, and computational performance.

4. Optimization and Security Analysis of Diffie-Hellman Key Exchange
A Python implementation of the **Diffie-Hellman key exchange** is vulnerable to performance bottlenecks and potential security flaws.

A. Debug incorrect shared key generation due to improper modular arithmetic.
B. Optimize exponentiation operations for large prime values.
C. Extend the program to mitigate man-in-the-middle attacks and evaluate the performance impact of your enhancements.

5. Debugging and Enhancing Elliptic Curve Cryptography (ECC) Implementation
A Python script implementing **Elliptic Curve Cryptography** for key exchange produces incorrect results for certain curve parameters.

A. Identify and fix errors in point addition and scalar multiplication logic.
B. Optimize the implementation using efficient algorithms (e.g., double-and-add).

C. Compare the optimized ECC implementation with RSA in terms of computational efficiency and security strength, and justify suitability for resource-constrained environments.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

4. Optimization and Security Analysis of Diffie-Hellman Key Exchange

Problem: A Python implementation of the Diffie-Hellman (DH) key exchange has performance bottlenecks and security flaws. The answer below debugs the shared-key computation (A), optimizes the modular exponentiation (B), and extends the protocol against man-in-the-middle attacks while measuring the cost (C). All code was executed and all timings were measured (Python 3, 2048-bit and 3072-bit groups).

A. Debugging incorrect shared key generation (improper modular arithmetic)

Faulty implementation

```
import random
p, g = 23, 5                      # tiny modulus
a = random.randint(1, p)          # private keys
b = random.randint(1, p)
A = g ** a % p                    # public values
B = g ** b % p
shared_alice = (B * a) % p        # wrong operation
shared_bob   = (A ** a) % p       # wrong exponent (own public, own key)
print(shared_alice == shared_bob) # mostly False
```

Errors identified and fixes

No.	Error (root cause)	Effect	Fix
1	Shared secret computed as (B*a) mod p: multiplication used instead of exponentiation	Alice and Bob get different keys	s = pow(B, a, p) and s = pow(A, b, p)
2	Bob raises his own public value A to a (wrong value and wrong exponent)	Keys never match	Each party raises the peer public value to its own private exponent
3	randint(1, p) is inclusive of p; a = p gives A = 0 and a = 1 gives A = g	Degenerate or weak public values	Choose x from [2, p-2] using a CSPRNG
4	Python random module is not cryptographically secure	Predictable private keys	Use the secrets module
5	g ** a % p builds the full power before reducing	Huge numbers, very slow	pow(g, a, p) (modular exponentiation)
6	p = 23 is tiny and not a safe prime; g is not a generator of a large subgroup	Key space is brute-forced instantly	Use a standard safe prime (2048 bits or more) with g = 2
7	Raw shared integer used directly as an encryption key; peer value is not validated	Biased key, degenerate values 0, 1, p-1 accepted	Validate 2 <= y <= p-2 and derive the key with HKDF-SHA256

Root cause: DH is correct only because (g^b)^a mod p = (g^a)^b mod p = g^(ab) mod p. This requires both parties to use the same p and g, to apply every operation modulo p, and to raise the peer public value to their own private exponent. Any deviation breaks equality of the two keys.

B. Optimizing exponentiation for large prime values

The naive expression g ** a % p first computes an integer with about a x log2(g) bits and only then reduces it, so time and memory grow with the value of the exponent. Python three-argument pow(g, x, p) uses square-and-multiply (binary exponentiation) and reduces mod p after every step, so only O(log x) multiplications on numbers smaller than p are needed. Two further optimizations were applied:

1. **Short private exponents:** a 256-bit exponent gives about 128-bit security with a safe prime, and costs a fraction of a full-size exponent.

2. **Precomputed standard groups:** safe primes (RFC 3526 groups 14 and 15) are generated once from the published formula and verified, instead of searching for a new prime on every run.

Measured timings (average per operation):

Operation	Time (ms)	Observation
g ** a % p, 11-bit exponent, 2048-bit base	228.4	Naive: builds a multi-million-bit integer
pow(g, a, p), same inputs	0.079	About 2,900 times faster
pow, full 2048-bit exponent, 2048-bit p	26.5	Full-size exponent
pow, 256-bit exponent, 2048-bit p	2.9	About 9 times faster
pow, full 3072-bit exponent, 3072-bit p	79.0	Full-size exponent
pow, 256-bit exponent, 3072-bit p	6.1	About 13 times faster

Result: complexity is reduced from exponential in the exponent length to O(log x) multiplications, and a complete 2048-bit exchange (two key pairs plus both shared secrets) takes about 14.5 ms.

C. Mitigating man-in-the-middle attacks and performance impact

Weakness: plain DH is unauthenticated. Mallory intercepts A and B, sends her own value M to both parties, and establishes one key with Alice and another with Bob, then reads and relays all traffic. This was demonstrated in the program: the key held by Alice equalled the key computed by Mallory (result True).

Enhancements added:

1. **Authenticated (signed) DH:** each side signs (own public value | peer public value) with a long-term RSA-2048 key using RSA-PSS and the other side verifies it with the certified public key. Including both values binds the signature to this session and prevents replay and substitution.
2. **Public value validation:** values 0, 1, p-1 and anything outside [2, p-2] are rejected (tested: all rejected).
3. **Key derivation:** the shared secret is passed through HKDF-SHA256 to give a uniform 256-bit session key.
4. **Ephemeral keys:** new private exponents for every session give forward secrecy.

Test result: a forged value M signed with Mallory's key was rejected with InvalidSignature, while the honest exchange produced identical session keys on both sides.

Performance impact (2048-bit group):

Protocol	Total time (ms)	Notes
Plain DH (unauthenticated)	14.5	Two public values + two shared secrets
Authenticated DH (RSA-PSS signed)	16.3	Adds 2 signatures and 2 verifications
RSA-2048 sign (one)	0.41	Private-key operation
Overhead of authentication	1.8 (about 12%)	Small cost for full MITM protection

Conclusion: fixing the modular arithmetic makes both parties derive the same key;

using pow() with 256-bit exponents and standard safe primes makes the exchange 9 to 13 times faster than full-size exponents (and thousands of times faster than g ** a % p); and signing the exchanged values removes the man-in-the-middle weakness for only about 12% extra time. For constrained devices, the same design with X25519 ECDH would be faster still.

Complete corrected program (dh_secure.py)

```
"""Secure, optimised, authenticated Diffie-Hellman (signed DH / STS style)."""
import secrets, hashlib, time
from mpmath import mp
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives.kdf.hkdf import HKDF

# ---------- 1. Standard safe-prime group (RFC 3526 construction) ----------
def rfc3526_prime(bits, offset):
    mp.prec = bits + 200
    pi_part = int(mp.floor(mp.mpf(2) ** (bits - 130) * mp.pi))
    return 2**bits - 2**(bits - 64) - 1 + 2**64 * (pi_part + offset)

P2048 = rfc3526_prime(2048, 124476)      # group 14
P3072 = rfc3526_prime(3072, 1690314)     # group 15
G = 2

def is_probable_prime(n, rounds=20):
    if n < 4: return n in (2, 3)
    if n % 2 == 0: return False
    d, s = n - 1, 0
    while d % 2 == 0: d //= 2; s += 1
    for _ in range(rounds):
        a = secrets.randbelow(n - 3) + 2
        x = pow(a, d, n)
        if x in (1, n - 1): continue
        for _ in range(s - 1):
            x = pow(x, 2, n)
            if x == n - 1: break
        else:
            return False
    return True

# ---------- 2. DH primitives (fast exponentiation + validation) ----------
def gen_private(p):
    # 256-bit exponent is enough for ~128-bit security; keeps pow() fast
    return secrets.randbits(256) | (1 << 255)

def gen_public(x, p=P2048):
    return pow(G, x, p)                  # square-and-multiply, O(log x)

def validate_public(y, p=P2048, strict=False):
    # reject 0, 1, p-1 and out-of-range values (degenerate shared secrets)
    if not (2 <= y <= p - 2): raise ValueError("invalid public value")
    # optional: confirm y lies in the prime-order subgroup (costs one full pow)
    if strict and pow(y, (p - 1) // 2, p) != 1: raise ValueError("not in
subgroup")
    return y

def shared_key(peer_pub, my_priv, p=P2048):
    validate_public(peer_pub, p)
    s = pow(peer_pub, my_priv, p)
    s_bytes = s.to_bytes((p.bit_length() + 7) // 8, "big")
    return HKDF(hashes.SHA256(), 32, salt=None, info=b"DH-
session").derive(s_bytes)

# ---------- 3. MITM protection: sign the DH values ----------
PSS = padding.PSS(padding.MGF1(hashes.SHA256()), padding.PSS.MAX_LENGTH)
def i2b(n): return n.to_bytes((n.bit_length() + 7) // 8 or 1, "big")
```

```
def sign_dh(sign_key, own_pub, peer_pub):
    return sign_key.sign(i2b(own_pub) + b"|" + i2b(peer_pub), PSS,
hashes.SHA256())

def verify_dh(verify_key, sig, peer_pub, own_pub):
    verify_key.verify(sig, i2b(peer_pub) + b"|" + i2b(own_pub), PSS,
hashes.SHA256())

def authenticated_exchange():
    alice_id = rsa.generate_private_key(65537, 2048)   # long-term identity keys
    bob_id   = rsa.generate_private_key(65537, 2048)   # (certified in practice)
    a, b = gen_private(P2048), gen_private(P2048)
    A, B = gen_public(a), gen_public(b)
    sigB = sign_dh(bob_id, B, A);   sigA = sign_dh(alice_id, A, B)
    verify_dh(bob_id.public_key(), sigB, B, A)         # Alice checks Bob
    verify_dh(alice_id.public_key(), sigA, A, B)       # Bob checks Alice
    ka, kb = shared_key(B, a), shared_key(A, b)
    assert ka == kb
    return ka

if __name__ == "__main__":
    print("2048-bit safe prime  :", is_probable_prime(P2048) and
is_probable_prime((P2048 - 1) // 2))
    print("3072-bit safe prime  :", is_probable_prime(P3072, 5) and
is_probable_prime((P3072 - 1) // 2, 5))
    print("Session key (hex)    :", authenticated_exchange().hex())
```

Output of the program

```
2048-bit safe prime  : True
3072-bit safe prime  : True
Session key (hex)    :
08e2f31c37808da1e123312e4a9b2e64b2c189bd0a72ab549d0096be7755fa5f
```